📘 DP Notes: Kadane's Algorithm & Maximum Subarray Sum with At Most One Deletion

Part 1: Kadane's Algorithm (Maximum Subarray Sum)

Problem Statement

Given an integer array `arr[]`, find the maximum sum of a **non-empty contiguous subarray**.

Example

```
arr = [-2,1,-3,4,-1,2,1,-5,4]
Answer = 6
Subarray = [4,-1,2,1]
```

Brute Force

Generate every subarray and calculate its sum.

```python
def brute(arr):
    n = len(arr)
    ans = float('-inf')

    for i in range(n):
        s = 0
        for j in range(i, n):
            s += arr[j]
            ans = max(ans, s)

    return ans
```

Complexity

```
Time  : O(N²)
Space : O(1)
```

DP State Building

Ask yourself:

> What information do I need at every index?

Define:

```
f(i)
```

= Maximum subarray sum **ending at index i**.

The phrase **ending at i** is extremely important.

Recurrence Logic Building

At index `i`, we have only two choices.

Choice 1: Start a new subarray

```
arr[i]
```

Example:

```
[4]
```

Choice 2: Extend previous subarray

```
arr[i] + f(i-1)
```

Example:

```
[4,-1,2]
```

Recurrence

```
f(i) = max(
    arr[i],
    arr[i] + f(i-1)
)
```

Base Case

```
f(0) = arr[0]
```

Recursive Solution

```python
def recursion(arr):
    n = len(arr)

    def f(i):
        if i == 0:
            return arr[0]

        return max(
            arr[i],
            arr[i] + f(i-1)
        )

    ans = float('-inf')

    for i in range(n):
        ans = max(ans, f(i))

    return ans
```

Complexity

```
Time  : O(N²)
Space : O(N)
```

Memoization

```python
def memoization(arr):
    n = len(arr)
    memo = {}

    def f(i):
        if i == 0:
            return arr[0]

        if i in memo:
            return memo[i]

        memo[i] = max(
            arr[i],
            arr[i] + f(i-1)
        )

        return memo[i]

    ans = float('-inf')

    for i in range(n):
        ans = max(ans, f(i))

    return ans
```

Complexity

```
Time  : O(N)
Space : O(N)
```

Tabulation

DP Table

```
dp[i]
```

= Maximum subarray sum ending at `i`.

Base Case

```
dp[0] = arr[0]
```

Transition

```
dp[i] = max(
    arr[i],
    arr[i] + dp[i-1]
)
```

Code

```
def tabulation(arr):
    n = len(arr)

    dp = [0] * n
    dp[0] = arr[0]

    ans = arr[0]

    for i in range(1, n):
        dp[i] = max(
            arr[i],
            arr[i] + dp[i-1]
        )

        ans = max(ans, dp[i])

    return ans
```

Space Optimization (Actual Kadane)

Observe:

```
dp[i]
```

depends only on:

```
dp[i-1]
```

Therefore, we only need one variable.

Code

```python
def kadane(arr):
    cur = arr[0]
    ans = arr[0]

    for i in range(1, len(arr)):
        cur = max(
            arr[i],
            arr[i] + cur
        )

        ans = max(ans, cur)

    return ans
```

Complexity

```
Time  : O(N)
Space : O(1)
```

Dry Run

```
arr = [4,-1,2,1]
```

i	arr[i]	cur	ans
0	4	4	4
1	-1	3	4
2	2	5	5

i	arr[i]	cur	ans
3	1	6	6

Answer = 6

Part 2: Maximum Subarray Sum with At Most One Deletion

Problem Statement

Given an array `arr[]`, find the maximum sum of a non-empty subarray.

You may delete **at most one element**.

The subarray after deletion must still be **non-empty**.

Example

```
arr = [1,-2,0,3]
```

Delete:

```
-2
```

Subarray becomes:

```
[1,0,3]
```

Answer:

```
4
```

Main Observation

Normal Kadane state:

```
f(i)
```

New condition:

```
one deletion allowed
```

So we add another state.

State Definition

```
f(i, deleted)
```

where

```
deleted = 0
```

means:

No deletion used.

```
deleted = 1
```

means:

One deletion already used.

State Meaning

State 1

```
f(i,0)
```

Maximum subarray sum ending at `i` without using deletion.

State 2

```
f(i,1)
```

Maximum subarray sum ending at `i` after using one deletion.

Recurrence for f(i,0)

Exactly Kadane.

```
f(i,0)=max(
    arr[i],
    arr[i]+f(i-1,0)
)
```

Recurrence for f(i,1)

This is the important part.

There are only two possibilities.

Case 1

Deletion happened earlier.

So we simply include current element.

```
arr[i]+f(i-1,1)
```

Case 2

Delete current element.

Then the subarray effectively ends at:

```
i-1
```

And till `i-1`, we haven't used deletion.

So:

```
f(i-1,0)
```

Final Recurrence

```
f(i,1)=max(
    arr[i]+f(i-1,1),
    f(i-1,0)
)
```

Base Cases

```
f(0,0)=arr[0]
```

```
f(0,1)=-inf
```

Why?

Deleting the only element creates an empty subarray.

Not allowed.

Recursive Solution

```
def recursion(arr):
    n = len(arr)

    def f(i, deleted):
```

```
        if i == 0:
            if deleted == 0:
                return arr[0]
            return float('-inf')

        if deleted == 0:
            return max(
                arr[i],
                arr[i] + f(i-1,0)
            )

        return max(
            arr[i] + f(i-1,1),
            f(i-1,0)
        )

    ans = float('-inf')

    for i in range(n):
        ans = max(
            ans,
            f(i,0),
            f(i,1)
        )

    return ans
```

Memoization

```
def memoization(arr):
    n = len(arr)
    memo = {}

    def f(i, deleted):

        if i == 0:
            if deleted == 0:
                return arr[0]
            return float('-inf')

        if (i, deleted) in memo:
            return memo[(i, deleted)]

        if deleted == 0:
            ans = max(
                arr[i],
                arr[i] + f(i-1,0)
            )
```

```
            else:
                ans = max(
                    arr[i] + f(i-1,1),
                    f(i-1,0)
                )

            memo[(i, deleted)] = ans
            return ans

        ans = float('-inf')

        for i in range(n):
            ans = max(
                ans,
                f(i,0),
                f(i,1)
            )

        return ans
```

Tabulation

DP Table

```
dp[i][0]
```

Maximum subarray sum ending at `i` without deletion.

```
dp[i][1]
```

Maximum subarray sum ending at `i` after using one deletion.

Table Size

```
dp = [[0]*2 for _ in range(n)]
```

Base Cases

```
dp[0][0]=arr[0]
dp[0][1]=float('-inf')
```

Transition

```
dp[i][0]=max(
    arr[i],
    arr[i]+dp[i-1][0]
)
```

```
dp[i][1]=max(
    arr[i]+dp[i-1][1],
    dp[i-1][0]
)
```

Iteration Order

Everything depends on:

```
i-1
```

Therefore:

```
for i in range(1,n):
```

Tabulation Code

```
def maximumSum(arr):
    n = len(arr)
```

```
    dp = [[0]*2 for _ in range(n)

    dp[0][0] = arr[0]
    dp[0][1] = float('-inf')

    ans = arr[0]

    for i in range(1,n):

        dp[i][0] = max(
            arr[i],
            arr[i]+dp[i-1][0]
        )

        dp[i][1] = max(
            arr[i]+dp[i-1][1],
            dp[i-1][0]
        )

        ans = max(
            ans,
            dp[i][0],
            dp[i][1]
        )

    return ans
```

Dry Run

```
arr=[1,-2,0,3]
```

i	dp[i][0]	dp[i][1]
0	1	-∞
1	-1	1
2	0	1
3	3	4

Answer:

```
4
```

Space Optimization

Observe:

```
dp[i][0]
```

depends only on:

```
dp[i-1][0]
```

```
dp[i][1]
```

depends only on:

```
dp[i-1][0]
dp[i-1][1]
```

Only previous row is needed.

Variables

```
noDel
```

represents:

```
dp[i-1][0]
```

```
oneDel
```

represents:

```
dp[i-1][1]
```

Why

```
oneDel = -inf
```

and not

```
arr[0]
```

Because:

```
dp[0][1]
```

means:

We already used one deletion.

Deleting the only element leaves an empty subarray.

Impossible state.

Therefore:

```
oneDel = -inf
```

Space Optimized Code

```
def maximumSum(arr):

    noDel = arr[0]
    oneDel = float('-inf')

    ans = arr[0]

    for i in range(1, len(arr)):
```

```
        newNoDel = max(
            arr[i],
            arr[i] + noDel
        )

        newOneDel = max(
            arr[i] + oneDel,
            noDel
        )

        noDel = newNoDel
        oneDel = newOneDel

        ans = max(
            ans,
            noDel,
            oneDel
        )

    return ans
```

Space Optimization Dry Run

```
arr = [1,-2,0,3]
```

Initial:

```
noDel = 1
oneDel = -inf
ans = 1
```

---

i = 1

```
newNoDel = -1
newOneDel = 1
```

---

i = 2

```
newNoDel = 0
```

```
newOneDel = 1
```

---

i = 3

```
newNoDel = 3
newOneDel = 4
```

Answer:

```
4
```

Pattern Learned Today

Pattern 1

```
Maximum Subarray
↓
Kadane DP
↓
Space Optimized DP
```

Pattern 2

```
Old DP State
+
One Extra Operation Allowed
↓
Add one Boolean State
```

Template:

```
f(i, used)
```

where

```
used = 0
used = 1
```

When to Think of This Pattern

Questions containing:

- at most one deletion
- at most one modification
- at most one flip
- at most one transaction
- one mistake allowed
- one operation allowed

General Template

```
dp[i][0]
```

operation not used.

```
dp[i][1]
```

operation already used.

Similar Problems

1. Maximum Subarray Sum with One Deletion
2. Best Time to Buy and Sell Stock III
3. Best Time to Buy and Sell Stock IV
4. Longest Subarray of 1's After Deleting One Element
5. Max Consecutive Ones II
6. Flip One Zero to Maximize Consecutive Ones
7. Maximum Sum Circular Subarray
8. Edit Distance Variants
9. Buy and Sell Stock with Cooldown
10. Buy and Sell Stock with Transaction Fee

Interview Tricks

Trick 1

If you see:

```
at most one operation
```

Immediately think:

```
Old DP state + Boolean State
```

Trick 2

If recurrence depends only on:

```
i-1
```

Immediately think:

```
Space Optimization Possible
```

Trick 3

Impossible states:

For maximization DP:

```
float('-inf')
```

For minimization DP:

```
float('inf')
```

Biggest Lesson Today

```
Kadane is NOT Sliding Window.

Kadane is:
Recursion
↓
Memoization
↓
Tabulation
↓
Space Optimized DP
```

And:

```
One extra operation allowed
↓
Add one Boolean State.
```